Министерство образования и науки Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего образования
«Иркутский государственный университет»
(ФГБОУ ВО «ИГУ»)
Факультет бизнес-коммуникаций и информатики

КУРСОВАЯ РАБОТА
Б1.В.20 Курсовая работа по профилю

по направлению 09.03.03 «Прикладная информатика»
(прикладной бакалавриат)
профиль «Прикладная информатика (Разработка программного обеспечения)»

Анализ возможностей современных инструментов разработки языков программирования

Студент 3 курса очной формы обучения, группа 14321-ДБ
_____ И.Д. Донской

Руководитель: ст. преподаватель
_____ А.В. Киселёв

Работа защищена:
«_____» _____ 2026 г.
С оценкой _____

Иркутск 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ ... 3
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ РАЗРАБОТКИ ЯЗЫКА ПРОГРАММИРОВАНИЯ ... 5
1.1 Сравнительный анализ существующих языков программирования ... 5
1.2 Методы анализа исходного кода программы ... 8
1.3 Процесс выполнения исходного кода программы ... 17
Выводы по главе ... 18
2 РАЗРАБОТКА ЯЗЫКА ПРОГРАММИРОВАНИЯ И ПРОТОТИПА ИНТЕРПРЕТАТОРА ... 20
2.1 Обоснование выбора инструментов и технологий ... 20
2.2 Архитектура интерпретатора ... 21
2.3 Описание языка программирования Aqua ... 23
2.4 Программная реализация интерпретатора ... 26
2.5 Интеграция языка Aqua со средой разработки ... 31
Выводы по главе ... 32
ЗАКЛЮЧЕНИЕ ... 33
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ... 36
ПРИЛОЖЕНИЯ ... 38
Приложение А Фрагменты исходного кода модуля frontend ... 38
Приложение Б Фрагменты исходного кода модуля runtime ... 39
Приложение В Примеры программ на языке Aqua ... 40
Приложение Г Консольные команды для запуска утилит ... 41
Приложение Д Реализация тестирования ... 42
Приложение Е Интеграция со средой разработки ... 43
Приложение Ж Установка языка Aqua ... 44
Приложение И Практические примеры использования ... 45

ВВЕДЕНИЕ

В современных условиях стремительного развития информационных технологий языки программирования играют ключевую роль в создании программного обеспечения различной сложности. Популярные языки программирования предлагают различные подходы и инструменты для написания кода, однако зачастую они имеют избыточный синтаксис, высокую сложность освоения, наследуют структурные недостатки предшественников и недостаточную выразительность в определённых предметных областях. Это затрудняет освоение начинающими разработчиками, выпуск новых квалифицированных кадров, усложняет поиск ошибок, увеличивает время разработки и отладки, и создаёт трудности при поддержке приложения. Постоянное развитие программных систем, приводят к росту требований к производительности и масштабируемости систем. Таким образом, разработка новых решений обретает особую **практическую значимость**.

Разработка нового языка программирования позволяет применить совершенно новые подходы в семантическом и синтаксическом анализе для решения практических задач. Таким образом, анализ современных инструментов разработки языков программирования нацелен на изучение внедрения новых подходов для дальнейшей разработки языка программирования, обладающим такими преимуществами, как: простота использования, хорошая читаемость, широкая функциональность и достаточная выразительность с поддержкой современных подходов программирования и автоматическим поиском ошибок на этапе анализа.

Актуальность данной работы обусловлена необходимостью анализа возможностей современных инструментов разработки языков программирования, а также изучения подходов к созданию интерпретаторов как одного из наиболее доступных способов реализации языков.

Целью данной работы является анализ возможностей современных инструментов разработки языков программирования и практическая реализация интерпретатора для учебного языка Aqua.

Для достижения поставленной цели необходимо решить следующие **задачи**.

1. Провести анализ существующих языков программирования.

2. Изучить архитектуру и инструменты, используемые в реализации языков программирования.

3. Разработать язык программирования Aqua и реализовать учебный прототип интерпретатора.

Объектом исследования являются инструменты разработки языков программирования, а также процессы создания интерпретаторов.

Предметом исследования выступают методы и средства реализации интерпретаторов, а именно: теория формальных языков, методы лексического, синтаксического и семантического анализа; применение генераторов анализаторов, таких как jflex и byacc/j и вспомогательных инструментов; проектирование структуры проекта для реализации интерпретатора; особенности реализации выполнения программ на разработанном языке Aqua.

В рамках данной работы используются следующие методы и подходы:

- аналитический метод — для изучения существующих языков программирования и инструментов их разработки;
- сравнительный анализ — для выявления преимуществ и недостатков различных подходов и технологий;
- метод проектирования программного обеспечения — при разработке архитектуры интерпретатора;
- экспериментальный метод — при реализации и тестировании разработанного интерпретатора.

В качестве практического подхода используется поэтапная разработка интерпретатора, включающая реализацию лексического, синтаксического и семантического анализа с последующей интерпретацией программы. Такой подход позволяет наглядно продемонстрировать полный цикл обработки исходного кода и оценить возможности современных инструментов разработки языков программирования.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ РАЗРАБОТКИ ЯЗЫКА ПРОГРАММИРОВАНИЯ

Языки программирования являются основным инструментов взаимодействия программиста с компьютером. За время развития отрасли появилось множество языков, отличающихся синтаксисом, семантикой, областью применения, способом исполнения кода, скоростью работы, принципами работы с памятью и многими другими аспектами, поэтому необходимо рассмотреть их преимущества, недостатки, а также пробелы в функциональности.

1.1 Сравнительный анализ существующих языков программирования

Разработка нового языка программирования требует предварительного анализа существующих решений, их преимуществ и недостатков. Такой анализ позволяет выявить ключевые проблемы современных языков и сформулировать требования к инструментам разработки компиляторов и интерпретаторов для устранения недостатков. Одной из важных задач является исследование возможности выполнения статического анализа программы до её запуска. В частности, представляет интерес выполнение следующих проверок:

- автоматическое определение типов;
- анализ достижимости кода и полноты ветвлений;
- возможность реализации различных видов сокращённых конструкций, например, для цикла for.

Реализация подобных механизмов позволяет проверить возможность реализации и изучить уровень гибкости, которого может достичь создаваемый язык программирования.

Классификация языков программирования

Современные языки программирования можно классифицировать по нескольким основным критериям. По способу выполнения существуют компилируемые (например, C/C++), интерпретируемые (например, Python), смешанные (с использованием виртуальной машины, например, Java). По

парадигме программирования языки делятся на императивные, функциональные, логические, объектно-ориентированные, мультипарадигменные. По системе типов разделение на статически типизированные и динамически типизированные, строго типизированные и слабо типизированные.

Статическая типизация позволяет выявлять ошибки на этапе компиляции или анализа, что повышает надёжность программ. В свою очередь, динамическая типизация обеспечивает большую гибкость и скорость разработки, но переносит часть ошибок на этап выполнения. Таким образом, выбор системы типов является компромиссом между безопасностью и удобством разработки.

Рассмотрим наиболее распространённые языки программирования и их характерные черты.

- C/C++ — обеспечивают высокую производительность и контроль над ресурсами, однако обладают сложным синтаксисом. Наличие небезопасных конструкций (например, работа с памятью [1]) приводит к частым ошибкам, а диагностические сообщения компилятора могут быть трудны для понимания [2].

- Java — предлагает строгую объектную модель, переносимость и хорошо структурированный код. Однако язык характеризуется избыточностью синтаксиса, что увеличивает объём кода [3, 4].

- Python — отличается простотой и лаконичностью, что делает его популярным для быстрого прототипирования [5]. При этом язык уступает по производительности [6] и не обеспечивает полноценной статической проверки типов [7].

- Go — сочетает строгий синтаксис и простоту, а также предоставляет удобные средства для разработки параллельных программ. Однако язык не всегда позволяет выразить сложные конструкции компактно [8].

- Rust — ориентирован на безопасность работы с памятью [9] и предотвращение ошибок на этапе компиляции [10]. Главным недостатком является высокая сложность освоения и трудность реализации комплексных структур.

Отдельно следует отметить, что такие языки, как C/C++, исторически развивались длительное время. Добавление нового функционала часто приводит к усложнению синтаксиса и появлению неоднородных решений, что затрудняет их использование и реализацию компиляторов.

Язык Python, напротив, получил широкое распространение благодаря простоте. Однако отсутствие встроенной строгой статической типизации привело к появлению внешних инструментов, таких как mypy, предназначенных для анализа типов. Это указывает на востребованность подобных механизмов, которые, по возможности, должны быть встроены в сам язык.

Анализ существующих языков позволяет выделить ряд общих недостатков:

- наличие конструкций, подверженных ошибкам (например, ручное управление памятью в C++);
- избыточность и громоздкость синтаксиса (Java, C++);
- высокая сложность освоения (C++, Rust);
- ограниченная производительность интерпретируемых языков (Python);
- недостаточная выразительность или лаконичность некоторых конструкций;
- отсутствие встроенных средств статического анализа в ряде языков.

Данные недостатки формируют основу для разработки нового языка, ориентированного на простоту, безопасность и выразительность.

Проведённый анализ показывает, что ни один из существующих языков не сочетает в полной мере комбинацию следующих характеристик: простоту синтаксиса, строгую и удобную систему типов, высокую производительность, встроенные средства статического анализа, безопасность, лаконичность записи конструкций.

Это подтверждает актуальность изучения инструментов разработки компиляторов и интерпретаторов, а также подталкивает к дальнейшему изучения вопроса о создании нового языка программирования.

На основе проведённого анализа можно сформулировать требования к разрабатываемому языку и инструментам его реализации:

- поддержка статического анализа: автоматическое определение типов переменных и функций, проверка корректности возвращаемых значений, анализ полноты ветвлений и наличия return;
- простота и лаконичность синтаксиса: минимизация служебных конструкций и поддержка сокращённых форм записи;
- безопасность выполнения: нахождение потенциально опасных конструкций на этапе анализа, контроль корректности операций, соответствие типов данных;
- гибкость конструкций управления: расширенные формы циклов, в частности, различные варианты for, поддержка выразительных управляющих структур;
- удобство реализации интерпретатора: формализуемая грамматика, возможность автоматической генерации лексера и парсера, простота построения синтаксического дерева.

Таким образом, сформированные требования определяют направление разработки языка Aqua и его интерпретатора, ориентированного на сочетание простоты, выразительности и надёжности.

1.2 Методы анализа исходного кода программы

Работа любого языка программирования начинается с считывания текста, представляющего собой исходный код программы, и последующее преобразование текста в удобную для анализа структуру. Зачастую этой структурой является дерево разбора. Это сложный процесс, поэтому его разбивают на несколько этапов, за которые отвечают разные подпрограммы. В общем случае порядок работы современного компилятора или интерпретатора выглядит следующим образом: считывание исходныого кода, лексический анализ (Lexer), синтаксический анализ (Parser), семантический анализ, генерация исполняемого файла / интерпретация [11, с. 32-34].

Первым этапом обработки исходного кода программы является лексический анализ. Можно сказать, что исходный код представляет собой массив символов. Для более удобной обработки, его необходимо разбить на части. Простое разделение по пробелам не подойдет, так как в программном коде, в отличии от обычного текста, есть арифметические и специальные символы, между которыми может даже не быть пробелов. Поэтому, для правильного разделения входных символов, на практике используют регулярные выражения. Для примера рассмотрим следующий код: «int x = 10;». Данный код отвечает за создание целочисленной переменной x со значением 10. Разбив данную строку, получим следующий результат: «int», «x», «=», «10», «;». В контексте лексического анализа данные последовательности символов называют лексемы [11, с. 158]. Рассмотрим аналогичный фрагмент кода, который создаёт переменную с плавающей точкой: «float num = 2.4;» Если разбить его на лексемы, получим последовательность: «float», «num», «=», «2.4», «;». В данных двух примерах мы можем заметить сходство: первым пишется тип переменной, затем название, знак равно, числовое значение, знак точка с запятой. Это можно записать в более общей форме, как показано на рисунке 1.1.

```
<тип 'float'> <название 'num'> <знак '='> <число '2.4'> <знак ';'>
```

Рисунок 1.1 – Представление в общей форме

Каждый такой элемент называется токеном. Он представляет собой категорию символа и хранит в себе исходный текст (лексему) [11, с. 158]. На рисунке 1.2 изображена запись, с использованием обозначений, которые используются на практике и в литературе.

```
int x=10; → <KEYWORD 'int'>, <ID 'x'>, <OPERATOR '='>, <NUMBER '10'>, <KEYWORD ';'>

float num=2.3; → <KEYWORD 'float'>, <ID 'num'>, <OPERATOR '='>, <NUMBER '2.4'>, <KEYWORD ';'>
```

Рисунок 1.2 – Представление в виде токенов

Зарезервированные в языке символы называют KEYWORD, названия переменных или функций - ID или IDENTIFIER, арифметические знаки OPERATOR, а числовые значения - NUMBER. Токены могут включать в себя дополнительную информацию [11, с. 159], такую как: категорию лексемы, номер

строки, номер столбца, имя файла. Обычно, чтобы разбить исходный текст на набор токенов, используют регулярные выражения (regex) [11, с. 168]. Например, IDENTIFIER можно представить регулярным выражением [a-zA-Z][a-zA-Z0-9]*, а NUMBER представить как [0-9]+.

Ошибка на этапе лексического анализа возникает при несоответствии входного символа ни одному из определений лексем [11, 161]. Например, код «string 2s = "Hello world"»; вызовет ошибку, поскольку название переменной (IDENTIFIER) не может начинаться с цифры. Также лексические ошибки могут возникать при использовании специфичных символов или кодировок.

Лексический анализ — это начальный этап обработки, на котором исходный код разбивается на последовательность лексем и преобразуется в набор токенов [11, с. 156]. Этот процесс позволяет структурировать текст программы и подготовить его для дальнейшего синтаксического анализа. За разбиение на токены и поиск ошибок отвечает лексический анализатор (лексер), используя регулярные выражения. В современных языках программный код лексера обычно не пишут вручную. Для этого используют кодогенерацию на основе заранее прописанной лексической грамматики. Один из самых популярных генераторов лексеров - flex. Он написан на C++, но позже на его основе появились реализации для других языков программирования [11, с. 191].

Синтаксический анализ (парсинг) — это этап обработки исходного кода, на котором набор токенов преобразуется в структурированное представление, которое называется абстрактным синтаксическим деревом (AST). На этом этапе проверяется соответствие программы правилам грамматики, выявляются ошибки, строится синтаксическое дерево [11, с. 252-253].

Понятие грамматики

Вернёмся к ранее упомянутые примеры и для удобства обозначим <KEYWORD float> как FLOAT, <KEYWORD int> как INT, <OPERATOR => как EQUAL, <KEYWORD ;> как SEMICOLON. Можно заметить, что полученные последовательности довольно схожи. Один из базовых подходов в

программировании и в разработке программного обеспечения — это абстракция и обобщение. Здесь мы тоже воспользуемся этим методом. В общем виде инициализация переменной будет иметь следующую структуру: TYPE ID EQUAL NUMBER SEMICOLON. Тип переменной — это TYPE, название переменных, как мы уже говорили, принято называть ID, оператор присваивания — EQUAL, число для инициализации — NUMBER, а точка с запятой — SEMICOLON [11, с. 260-261]. Большинство элементов здесь представляют собой простые токены, но TYPE является составным, поскольку состоит из FLOAT, либо INT. Запишем для него определение: TYPE: FLOAT | INT. Двоеточие означает «является», а знак «|» означает «или». В теории формальных языков и грамматик, данная запись называется правилом производства (продукция), где атомарные элементы (токены) называются терминалами, составные – нетерминалами [11, с. 258-260]. Первый нетерминал, с. которого начинается самое первое правило производства, называется начальным символом и обозначается он как S или Start. Таким образом, в примере выше, FLOAT и INT являются терминалами, а TYPE - нетерминалом.

Рассмотрим более сложный пример: «int n = 2 + 2 – 1». Теперь знак точки с запятой в конце является необязательным, а вместо числа можно написать арифметическое выражение без скобок со знаками плюс или минус. Соответствующий фрагмент грамматики для инициализации переменной представлен на рисунке 1.3.

```
VAR_INIT: TYPE ID ASSIGN EXPRESSION LINE_END
TYPE: FLOAT | INT
EXPRESSION: NUMBER MATH_OP EXPRESSION | NUMBER
MATH_OP: PLUS | MINUS
LINE_END: SEMICOLON |
```

Рисунок 1.3 – Фрагмент грамматики

VAR_INIT — это продукция инициализации переменной, TYPE — тип, EXPRESSION — арифметическое выражение, MATH_OP — математический

оператор плюс или минус, LINE_END — конец строки. В данном примере нетерминал LINE_END порождается терминалом SEMICOLON или может вовсе отсутствовать. Нетерминал EXPRESSION в своём определении имеет рекурсивный вызов, что является ключевым принципом в построении грамматик и позволяет описывать конструкции произвольной длины.

Как и код лексера, код синтаксического анализатора (парсера) генерируется на основе грамматики, описанных в таком же формате, как было описано в примерах выше. Одним из самых популярных инструментов для генерации парсера является byacc. На его основе также существуют аналоги для других языков программирования, например byacc/j [11, с. 363]. Таким образом, создание любого языка сводится к описанию правил производства, то есть к грамматике.

Рассмотрим основные классы грамматик, их особенности и область применения. В классификации, предложенной Ноаном Хомским, грамматики делят на 4 вида.

Неограниченные грамматики — это общий класс грамматик, где все остальные являются его частными случаями. Правила имеют вид: $\alpha \rightarrow \beta$, где α — непустая строка символов, β — любая строка. Такие грамматики используют редко, поскольку в них есть возможность поместить несколько нетерминалов в левой части правила производства, из-за чего они обладают высокой сложностью.

Контекстно-зависимые грамматики имеют общий вид вид: $\alpha A \beta \rightarrow \alpha \gamma \beta$, где α, β — любая строка символов, A — нетерминал, γ — непустая строка. Как следует из названия, применение правил производства в таких грамматиках зависит от окружения. Если говорить более простым языком, происходит замена A на γ. Особенностью является то, что длинна получившейся строки после применения продукции не должна уменьшаться. Данные грамматики всё ещё обладают высокой сложностью, поэтому применяются ограниченно.

Контекстно-свободные грамматики имеют вид: $A \rightarrow \gamma$, где A — нетерминал, γ — любая строка символов. Поскольку в левой части производства находится

всего один нетерминал, контекстно-свободные грамматики обладают не такой высокой сложностью и легко обрабатываются программно. Широко применяются для описания синтаксиса языков программирования [11, с. 258]. Пример контекстно-свободной грамматики представлен на рисунке 1.4.

```
Expr → Expr + Term
Expr → Term
Term → number
```

Рисунок 1.4 — Пример грамматики

Регулярные грамматики имеют вид: A→aB, где A, B — нетерминалы, a — терминал. Данный вид является наиболее простым классом грамматик. Широко используются в поиске и шаблонах благодаря регулярным выражениям. На этапе лексического анализа с помощью регулярных выражений описываются идентификаторы, числа, ключевые слова и так далее.

Дерево разбора — представление исходного кода программы в виде структуры данных «дерево», узлы которого представляют собой нетерминальные и терминальные символы, и соответствуют грамматике языка [12, 115]. Есть два основных вида деревьев разбора.

Полное дерево разбора (Parse Tree, Concrete syntax tree) полностью отражает грамматику языка и содержит все нетерминальные и терминальные символы [12, 115-116]. Полные деревья разбора почти не используются, поскольку содержат много избыточной информации (например, скобки, служебные правила).

Абстрактное синтаксическое дерево (AST, Abstract Syntax Tree) представляет собой упрощённое полное дерево разбора. Оно лишено ненужных деталей, а часть узлов сокращены. Остаются лишь значимые символы, которые в дальнейшем пригодятся для обработки. Такие элементы, как скобки, удаляются, поскольку порядок выполнения и принадлежность символов к конкретному выражению реализуется за счёт древовидной структуры хранения данных [12, 117]. На практике, при обработке языков используют именно AST,

что даёт ряд преимуществ: компактность хранения, ускорение и упрощение обхода дерева, легкость выполнения модификаций. Создание AST реализуют либо методом удаления узлов из полного синтаксического дерева, либо не значимые символы изначально игнорируются на этапе обработки цепочек грамматики.

Синтаксический анализатор (парсер) работает на основе контекстно-свободной грамматики, получает на вход результат лексического анализа, а в качестве результата выдаёт дерево разбора. По принципу работы существует два основных вида анализаторов.

LL-парсер - это нисходящий парсер [11, с. 283], который читает вход слева направо (L — left-to-right) и строит левый вывод (L — leftmost derivation). Наиболее распространённый вариант — LL(1), который использует 1 символ предпросмотра.

LR-парсер — это восходящий парсер [11, с. 302], который читает вход слева направо и строит правый вывод в обратном порядке. LR-парсеры работают по принципу: свёртки (reduce) последовательности символов к нетерминалам. Существуют его различные модификации.

- LR(0) — простой, но слабый.
- SLR(1) — улучшенный.
- LALR(1) — компромисс (используется в Yacc).
- LR(1) — самый мощный.

Ошибки на этапе синтаксического анализа возникают при несоответствии входного потока символов ни одному из определений правил производства грамматики [11, с. 254-255]. Например, код: string s = "Hello" "world"; вызовет ошибку, поскольку между двумя строками нет знака действия или оператора. В другом примере: string s = "Hello world" можно увидеть ошибку, знакомую многим программистам - пропущен знак точка с запятой. Стоит отметить, что на этом этапе проверяется только соответствие правилам грамматики, поэтому код float f = "Hello" / 5; на данном этапе будет корректным.

Синтаксический анализатор отвечает за преобразование последовательности токенов в структурированное представление программы — абстрактное синтаксическое дерево. Именно на этом уровне формируется понимание структуры кода в соответствии с заданной грамматикой, а также выявляются синтаксические ошибки. Грамматика выступает основой языка, позволяя описывать допустимые конструкции через правила производства. Выбор конкретного метода синтаксического анализа (LL или LR) зависит от требований к языку и сложности грамматики, однако оба подхода широко применяются в современных программах. Таким образом, синтаксический анализатор стоит дерево разбора, которое за тем передаёт на следующий этап обработки - семантический анализ.

После построения абстрактного синтаксического дерева следующим этапом является семантический анализ. Его цель — выявить смысловые ошибки и дополнить дерево необходимой информацией посредством его обхода и анализа [12, с. 147].

Таблица символов — это структура данных, которая хранит информацию об идентификаторах в программе: имя, тип, значение, ссылка на экземпляр родительского элемента и другие дополнительные атрибуты [12, с. 148]. Во время обхода дерева разбора, при обработке узла с оператором присвоения значения, анализатор взаимодействует таблицу символов. Если это первая инициализация идентификатора — добавляется новая запись в таблицу символов, если повторная — вызывается ошибка. При присвоении значения к идентификатору проверяется, чтобы значение уже присутствовало в таблице символов, то есть уже было инициализировано ранее [12, с. 160].

Область видимости определяет, в каком месте программы и к какому именно идентификатору осуществляется доступ. Основные виды областей видимости:

- глобальная (за пределами функций);
- локальная (внутри функций);
- блочная (внутри блоков if, while и подобных).

Зачастую допускается объявление идентификаторов с одинаковым именем в разных или вложенных областях видимости. Один из способов проверки их корректности осуществляется с помощью иерархической структуры в таблице символов. То есть при обращении к идентификатору в таблице символов ищется записать с соответствующим именем, если такой записи не нашлось, то поиск продолжается в родительской таблице [12, с. 151].

Проверка типов включает в себя проверку совместимости типов в выражениях, в присвоениях, передаваемых аргументах функции и возвращаемых значений функции. В выражениях типы данных проверяются, основываясь на типе идентификатора, указанном в таблице символов и на основе литералов, которые строго задают тип значения. В большинстве языков программирования допускается автоприведение типов, когда среди допустимых операций с разными типами данных итоговый тип выражения выбирается исходя из заранее заданных приоритетов. Например, при сложении чисел с типами данных float и int результат будет float, поскольку у типа данных float приоритет выше [12, с. 179].

После выполнения проверок, для упрощения последующей обработки, в AST добавляется дополнительная информация [12, с. 152], например: типы выражений, ссылки на записи в таблице символов, информация о преобразованиях типов.

Ошибки на этапе семантического анализа возникают при несоответствии смысловой корректности программы требованиям языка. Проверки на несоответствия в программном коде не генерируются утилитами, используемыми на предыдущих этапах. Рассмотрим основные виды семантических ошибок:

- несоответствие типов данных;
- использование необъявленных идентификаторов;
- повторное объявление переменных в одной области видимости;
- несоответствие количества или типов аргументов функции;
- применение неподдерживаемых операторов.

Например, код: int s = "Hello world"; вызовет ошибку, поскольку к переменной с типом int присваивается значение с типом string.

Семантический анализ является важным этапом обработки программы, потому как от него зависит преждевременное нахождение смысловых ошибок, связанных с неправильным использованием конструкций языка, таких как несоответствие типов, нарушение областей видимости и некорректное обращение к идентификаторам.

Степень сложности обнаруживаемых ошибок, ограничивается только реализацией анализатора и может включать и более сложные сценарии. Результатом данного этапа является абстрактное синтаксическое дерево, дополненное информацией необходимой для дальнейшего выполнения программы и её оптимизации.

1.3 Процесс выполнения исходного кода программы

Следующим этапом обработки является использование полученного абстрактного синтаксического дерева. Строго говоря, существует лишь два последующих пути обработки: генерация кода (машинного кода или кода другого языка программирования) и выполнение программы (интерпретация). В данном исследовании будет рассмотрен только процесс интерпретации. Используя алгоритм обхода дерева в глубину, программа посещает все узлы и выполняет действия в соответствии с типом узла и сохранённой в нём информацией [12, с. 215].

Обход дерева чаще всего реализуется рекурсивно. Для каждого типа узла определяется соответствующая логика выполнения:

- выражения вычисляются и возвращают значение;
- операторы присваивания изменяют состояние таблицы символов;
- условные конструкции выбирают ветку выполнения;
- циклы повторяют выполнение поддерева до выполнения условия выхода.

В процессе интерпретации ключевую роль играет таблица символов. При обработке узлов дерева интерпретатор обращается к этой таблице для чтения и обновления значений. Именно в ней хранится состояние всей программы.

К ошибкам во время выполнения относятся все ошибки, которые не удалось выявить на предыдущих этапах. Чаще всего это ошибки, связанные со значениями самих переменных.

На рисунке 1.5 приведён пример кода, который вызовет ошибку во время выполнения «деление на ноль».

```
int a = 24;
int b = a / 2 - 12;
int r = 5 / b;
```

Рисунок 1.5 – Пример кода с ошибкой

На этапе семантического анализа подобные ошибки трудно выявить, поскольку, для определения значений переменных, они требуют исполнения кода программы на этапе анализа. Но зачастую проверки и вовсе невозможны, поскольку данные приходят из вне.

Важно отметить, что интерпретатор выполняет программу на ходу, без предварительной генерации машинного кода. Это упрощает реализацию и повышает переносимость, однако уступает компилируемым программам в производительности.

Выводы по главе

Проведённый сравнительный анализ существующих языков программирования показал, что ни один из них в полной мере не сочетает простоту синтаксиса, строгую систему типов, высокую производительность и встроенные средства статического анализа, что обосновывает актуальность разработки нового языка. На основе выявленных недостатков современных языков были сформулированы ключевые требования к разрабатываемому языку: поддержка статического анализа, включая автоматическое определение типов и анализ полноты ветвлений, безопасность выполнения, гибкость конструкций управления и лаконичность синтаксиса. В работе описано, что процесс обработки исходного кода традиционно делится на этапы: лексический анализ, в который входит разбиение на токены с помощью регулярных выражений;

синтаксический анализ в процессе которого происходит построение AST на основе контекстно-свободной грамматики; семантический анализ производятся проверки смысловых ошибок. Лексический анализ преобразует исходный код в последовательность токенов, для автоматизации этого процесса применяются генераторы лексеров (например, flex). Синтаксический анализ, работающий на основе грамматики, преобразует поток токенов в абстрактное синтаксическое дерево (AST), которое лишено избыточных деталей полного дерева разбора и удобно для дальнейшей обработки. Семантический анализ с использованием таблицы символов и правил областей видимости выявляет смысловые ошибки, например, несоответствие типов или повторное объявление переменных, дополняя AST информацией, необходимой для последующей интерпретации. Заключительным этапом является интерпретация, где выполняется рекурсивный обход AST в глубину: интерпретатор вычисляет выражения, управляет состоянием таблицы символов и реализует логику условных конструкций и циклов, при этом ошибки этого этапа, например, деление на ноль, обнаруживаются непосредственно во время выполнения.

2 РАЗРАБОТКА ЯЗЫКА ПРОГРАММИРОВАНИЯ И ПРОТОТИПА ИНТЕРПРЕТАТОРА

2.1 Обоснование выбора инструментов и технологий

Для реализации интерпретатора языка программирования Aqua был выбран ряд инструментов, обеспечивающих производительность, удобство разработки и расширяемость системы.

В качестве основного языка реализации выбран Java. Данный выбор обусловлен рядом факторов. Java имеет богатую стандартную библиотеку, относительно простой синтаксис, автоматическое управление памятью, следует объектно-ориентированной парадигме [13]. Всё это упрощает структурирование компонентов интерпретатора таких как: лексический анализатор, парсер, семантический анализатор, внутренние структуры данных интерпретатора. Также, Java работает на виртуальной машине, что позволяет запускать код на разных операционных системах без перекомпиляции.

Дополнительным преимуществом является наличие развитых инструментов разработки. В частности, для Java широко используется профессиональная интегрированная среда разработки IntelliJ IDEA, которая предоставляет мощные средства навигации по коду, рефакторинга, отладки и анализа программ [14]. Использование данной среды существенно повышает продуктивность разработки и упрощает сопровождение проекта.

Для реализации лексического анализатора был выбран JFlex [15]. Он представляет собой генератор лексических анализаторов, аналогичный утилите Lex [16], но ориентированный на Java. Инструмент позволяет описывать правила токенизации с использованием регулярных выражений, автоматически генерируя эффективный код.

Выбор JFlex обусловлен рядом преимуществ по сравнению с альтернативами, а именно: в отличие от Lex и Flex, JFlex изначально ориентирован на Java и не требует интеграции с нативным кодом; по сравнению с ANTLR [17], JFlex предоставляет более низкоуровневый и предсказуемый контроль над процессом лексического анализа, что важно для учебного

интерпретатора; генерируемый код отличается высокой производительностью и простотой интеграции; в сравнении с ручной реализацией, это уменьшает время разработки и количество ошибок.

Синтаксический анализ реализован с использованием BYACC/J [18]. Данный инструмент является адаптацией классического генератора парсеров YACC [19] для языка Java. Он позволяет задавать контекстно-свободную грамматику языка и автоматически генерировать LALR(1)-парсер. Использование BYACC/J обосновано следующими факторами:

- инструмент реализует классический подход LALR(1), который широко применяется в компиляторах и часто используется в литературе;
- по сравнению с ANTLR, BYACC/J использует более традиционную модель восходящего анализа, что делает его ближе к классическим учебным материалам по теории компиляции;
- BYACC/J генерирует компактный и предсказуемый код без сложной рантайм-инфраструктуры;
- по сравнению с ручной реализацией, использование генератора парсеров снижает вероятность ошибок и упрощает разработку.

Альтернативные решения, такие как ANTLR [17] является более функциональным и сложным инструментом, однако он скрывает детали работы парсера. В рамках данной работы предпочтение было отдано BYACC/J, так как он позволяет глубже изучить процесс синтаксического анализа и лучше контролировать его поведение.

Таким образом, выбранные инструменты позволяют эффективно реализовать интерпретатор, обеспечивая удобство разработки, автоматизацию, отладки системы и избавляя от части потенциальных ошибок.

2.2 Архитектура интерпретатора

Программная реализация интерпретатора языка Aqua организована в виде модульной структуры, разделённой на два основных уровня: фронтенд (анализ исходного кода) и рантайм (выполнение программы). Все исходные файлы

располагаются в директории /src, что соответствует стандартной практике организации Java-проектов.

Точкой входа в приложение является файл Main.java. Данный класс содержит минимальную логику и выполняет делегирование управления методу main() класса j0. Такой подход позволяет изолировать пользовательский интерфейс запуска программы от внутренней логики интерпретатора и упрощает дальнейшую модификацию архитектуры.

Код в директории frontend (Приложение А) отвечает за обработку исходного текста программы и преобразование его в синтаксическое дерево.

- j0.java — центральный класс, который координирует процессы: инициализирует лексер, запускает парсер и управляет потоком обработки входных данных.
- Yylex.java — лексический анализатор, сгенерированный с помощью JFlex. Выполняет разбиение входного потока символов на последовательность токенов.
- Parser.java — синтаксический анализатор, сгенерированный с использованием BYACC/J. На основе заданной грамматики строит дерево разбора.
- Token.java — структура, представляющая токен. Содержит тип токена, его текстовое представление и номер строки, что необходимо для диагностики ошибок.
- Tree.java — реализация узла синтаксического дерева. Включает символ грамматики, список дочерних узлов и связанные токены.
- ParserVal.java — вспомогательная структура, используемая парсером для передачи значений между правилами грамматики.
- Serial.java — генератор уникальных идентификаторов, применяемый для маркировки узлов дерева и других объектов.
- Yyerror.java — модуль обработки синтаксических ошибок, обеспечивающий информативные сообщения при некорректном вводе.

Таким образом, пакет frontend реализует полный цикл преобразования исходного текста программы в абстрактное синтаксическое дерево, которое затем передаётся в пакет runtime (Приложение Б), который отвечает за выполнение программы и классы необходимые для обработки узлов абстрактного синтаксического дерева на этапе выполнения. Рассмотрим классы пакета runtime и их назначение.

- Interpreter.java — ключевой компонент системы. Выполняет обход синтаксического дерева, проводит семантический анализ и интерпретирует конструкции языка.
- SymbolTable.java — реализация таблицы символов. Обеспечивает хранение переменных, их значений и поддерживает вложенные области видимости (scope).
- RuntimeValue.java — универсальная обёртка для значений времени выполнения, содержащая тип и соответствующее значение.
- ValueType.java — класс с набором констант, определяющих возможные типы данных в языке (например, int, string и др.).
- PrimitiveHandler.java — модуль обработки примитивных операций, таких как арифметические вычисления и операции сравнения.
- FunctionHandler.java — отвечает за обработку вызовов функций, включая передачу аргументов и возврат значений.
- FunctionType.java — структура, описывающая сигнатуру функции (параметры, тип возвращаемого значения).
- ErrorHandler.java — компонент обработки ошибок времени выполнения, обеспечивающий корректное завершение программы и вывод диагностической информации.

2.3 Описание языка программирования Aqua

Язык программирования Aqua представляет собой процедурно-ориентированный интерпретируемый язык с синтаксисом, близким к языкам семейства C/Java, но упрощённым для учебных целей. Программа на Aqua

состоит из набора операторов и функций, где точкой входа обычно является сам скрипт.

Синтаксис языка характеризуется следующими особенностями:

- использование фигурных скобок {} для определения блоков кода;
- опциональное использование точки с запятой в конце выражений;
- возможность записи условий без обязательных круглых скобок;
- поддержка однострочных конструкций без явного блока.

Идентификаторы формируются по стандартным правилам: начинаются с буквы или символа подчёркивания и могут содержать буквы, цифры и знак нижнего подчёркивания. Язык чувствителен к регистру.

Семантика языка основана на интерпретации абстрактного синтаксического дерева. Все выражения вычисляются последовательно, а управление потоком осуществляется через стандартные конструкции (условия, циклы, функции). Области видимости реализуются с помощью вложенных таблиц символов, что позволяет корректно обрабатывать локальные переменные и параметры функций.

Типы данных и конструкции управления.

Aqua поддерживает ограниченный, но достаточный для исследовательских задач набор типов данных:

- int — целые числа;
- float — числа с плавающей точкой;
- bool — логические значения;
- intarray — массивы целых чисел.

Типизация статическая: тип переменной задаётся при объявлении или вычисляется автоматически и должен соблюдаться при использовании.

Язык поддерживает стандартный набор операций: арифметические (+, -, *, /, %), логические (&&, ||, !), операции сравнения (<, >, ==, !=), операции присваивания и сокращённые формы (+=, ++).

Ключевые конструкции управления: if, else if, else, while, do-while, расширенный for с несколькими вариантами синтаксиса. Управляющие операторы: break, continue, return.

Функции объявляются с помощью ключевого слова fn и могут принимать параметры и возвращать значения. Поддерживается рекурсия.

Язык Aqua обладает рядом особенностей, отличающих его от распространённых языков программирования.

- Упрощённый синтаксис: необязательные точки с запятой, опциональные скобки в условиях, опциональный тип данных при объявлении переменной, компактные формы записи циклов.
- Расширенный синтаксис цикла for: поддержка нескольких форм записи (короткая, диапазонная, полная), возможность задания шага и границ в декларативной форме.
- Автоопределение типов: возвращаемый тип функции определяется автоматически, тип при инициализации определяется за счёт литералов.

По сравнению с Java язык Aqua проще и менее формализован (например, отсутствует обязательная объектная модель). В отличие от Python, Aqua использует статическую типизацию и фигурные скобки вместо отступов. По сравнению с C/C++ отсутствуют указатели и ручное управление памятью.

Ограничения языка включают отсутствие объектно-ориентированного программирования, обработки исключений, обобщённых типов, стандартной библиотеки.

В рамках данной работы приведены типовые примеры программ на языке Aqua (приложение B), демонстрирующие возможности языка. Представленные примеры демонстрируют основные возможности языка: работу с типами данных, использование управляющих конструкций и циклов, использование стандартных функций, определение функций, реализация рекурсии, создание массива и обработка массива. Благодаря своей простоте и наглядности, эти примеры позволяют быстро освоить синтаксис и ключевые концепции языка.

2.4 Программная реализация интерпретатора

Реализация лексического анализатора

Правила разбора на лексемы задаются в файле javalex.l с использованием регулярных выражений. Каждому регулярному выражению соответствует код, который должен выполниться при считывании — вызов метода scan() с передачей кода токена или выполнение вспомогательных функций, например, увеличить номер строки.

Рассмотрим основные группы токенов.

- Ключевые слова – "int", "float", "if", "else", "while", "for", "fn", "return".
- Идентификаторы – [a-zA-Z_][a-zA-Z0-9_]*.
- Числовые литералы – [0-9]*"."[0-9]*([eE][+-]?[0-9]+)?.
- Строковые литералы – \"([^\"]|(\\.))*\".
- Операторы: арифметические (+, -, *, /), логические: (&&, ||), сравнения (==, !=, <, >), составные (+=, ++, --).
- Игнорируемые элементы: пробельные символы, комментарии (//, /* */).

Лексер в интерпретаторе Aqua реализован с использованием генератора JFlex (Приложение Г), который на основе регулярных выражений, формирует класс Yylex.java. Основным методом класса является yylex(). Данный метод считывает символы из входного потока, сопоставляет их с шаблонами регулярных выражений и для каждой лексемы выполняет код, который был указан в файле javalex.l.

После перехода из метода yylex() в метод scan() класса j0, начинается создание объекта класса Token. В конструктор этого класса передаются: код токена, переданный в виде параметра; текст лексемы, получаемый из метода Yylex.yytext(); номер строки - Yylex.yylineno. Затем токен передаётся парсеру через поле j0.par.yylval.

Лексический анализатор включает базовую обработку ошибок. В случае обнаружения неизвестного символа вызывается метод yyerror класса Yyerror, формирующий сообщение об ошибке. Пример ошибки: «Lexer error: unrecognized character @ at line 1».

Подводя итог, процессы на данном этапе можно описать следующим образом: сперва исходный файл считывается как поток символов, лексер последовательно анализирует входной поток, затем распознанные последовательности символов преобразуются в токены и, наконец, полученные токены передаются синтаксическому анализатору.

Реализация синтаксического анализатора

Правила синтаксического разбора задаются в файле j0gram.y. В фигурных скобках после правил прописаны семантические действия, которые указывают, какие из токенов попадут в дерево разбора, а какие будут проигнорированы. На рисунке 2.1 приведены основные группы конструкций.

Описание блоков кода и методов

```
Stmt: Block | ';' | ExprStmt | BreakStmt | ReturnStmt
      | IfStmt | IfElseStmt | MethodDecl
      | DoWhileStmt | WhileStmt | ForStmt | LocalVarDeclStmt;
MethodDecl: FN IDENTIFIER '(' FormalParmList ')' Block { ... }
```

Описание переменных и типов

```
FieldDecl: Type VarDecls { ... }
Type: INT | FLOAT | BOOL | STRING | Name | INTARRAY
```

Рисунок 2.1 – Основные группы конструкций

С помощью BYACC/J на основе файла грамматики автоматически генерируется класс Parser.java. Ключевым методом является yyparse(). Внутри себя данный метод вызывает функцию yylex() для получения следующего для обработки токена, а затем сопоставляет значение с заданными правилами производства. Для передачи значений между правилами используется класс ParserVal, который, по своей сути, является контейнером для объекта Token. Токены помещаются в объекты класса Tree, которые реализуют узлы дерева разбора.

При обнаружении синтаксической ошибки вызывается метод yyerror класса Yyerror, формирующий сообщение с указанием строки и типа ошибки.

Пример ошибки: «Parser error: unexpected token } at line 10».

Сформированные на предыдущем этапе токены передаются парсеру, который, используя правила производства указанные в файле грамматики j0gram.y, строит синтаксическое дерево и сообщает об ошибках. Класс j0.java, в свою очередь, выполняет роль координатора, вызывая методы обоих анализаторов.

Полученное на предыдущем этапе абстрактное синтаксическое дерево подлежит анализу на предмет ошибок и для вычисления служебной информации. Этот функционал реализует метод semantic() класса Interpreter. В качестве параметров он принимает дерево разбора и глобальную область видимости, в которой уже заданы некоторые значения, например, встроенные функции. Работа семантического анализатора, начинается с вызова analyzeBlock для корня дерева разбора. Данный метод рекурсивно обходит дерево, выполняя операции, соответствующие типу узла. При объявлении переменных добавляет новые записи в SymbolTable, проверяет их на повтор. Для присваивания проверяет существование переменной, типы, допустимость операций. Для выражений вызывает функцию analyzeExpr(), которая вычисляет тип результата, вызывает обработчики операций. При вызове функции проверяет количество и типы аргументов, создает новую область видимости.

В основе реализации SymbolTable стоит структура данных словарь, где ключом является имя переменной, которому соответствует её текущее состояние. В Java такая структура объявляется как HashMap<String, RuntimeValue>. RuntimeValue.java представляет собой универсальный контейнер для значений разных типов (int, float, bool, массивы, функции), который содержит тип переменной и само значение. Помимо этого, таблица символов поддерживает вложенность областей видимости через ссылку на родителя. Рассмотрим основные методы класса. Функция addVar(String name, RuntimeValue value) добавляет переменную, проверяет на повтор, getVar(String name) ищет переменную в текущей и родительских областях, contains(String name) проверяет наличие переменной. По завершении

обработки, ссылка на SymbolTable сохраняется в соответствующем узле самого синтаксического дерева.

Класс PrimitiveHandler отвечает за проверку типов операндов, автоматическое приведение к нужному типу и реализацию базовых операций над типами, а именно: арифметические, логические, сравнение.

Обработка функций осуществляется в классе FunctionHandler. При вызове функции он находит её в таблице символов, проверяет аргументы, создает новую область видимости, выполняет тело функции. Также класс имеет методы для обработки встроенных функций, такие как: добавление их в область видимости и вызов их реализации, написанной на Java.

Обработка ошибок реализуется в классе ErrorHandler. При возникновении нештатной ситуации вызывается один из методов класса, который, в свою очередь, выбрасывает исключение, обработка которого находится на уровень выше, в классе j0.

Семантический анализатор гарантирует корректность программы на уровне типов, областей видимости, операций и вызовов функций, предотвращая ошибки времени выполнения.

Метод eval() класса Interpreter отвечает за вычисления узлов дерева и выполнение исходной программы. Как и в семантическом анализе, здесь используется схожий принцип обхода дерева разбора, но вместо анализа узлов дерева происходит их исполнение. Стоит отметить, что на данном этапе не производится проверка инициализации переменных, областей видимости и типов. Благодаря выполненному семантическому анализу обеспечивается корректная работа программы без необходимости вычислительных затрат на проверку ошибок.

Функция eval() выполняет различные действия в зависимости типа обрабатываемого узла. Для блоков создает новую область видимости, интерпретирует все инструкции по порядку. Для конструкций присваивания вычисляет выражение и сохраняет результат в таблице символов. Для выражений вызывает evalExpr которое, возвращает вычисленное значение. При

вызове функций создает новую область видимости для локальных переменных, передаёт аргументы в функцию FunctionHandler.eval(), которая возвращает полученной значение, переданное в теле функции с помощью оператора return. Для встроенных функций вызывается реализация, написанная на языке Java.

Перечень встроенных функций и их назначение:

- print(...) — выводит значения аргументов в одну строку через пробел;
- println(...) — выводит значения аргументов в одну строку через пробел и переводит строку;
- printChar(int code) — выводит символ по его числовому коду;
- readInt() — читает целое число с ввода;
- readFloat() — читает число с плавающей точкой с ввода;
- newIntArray(int size) — создает новый массив целых чисел заданной длины;
- readCharsAsIntArray() — читает все строки ввода, возвращает массив кодов символов;
- readChar() — читает один символ с ввода, возвращает его числовой код;
- getElement(intarray arr, int index) — возвращает элемент массива по индексу;
- setElement(intarray arr, int index, int value) — устанавливает значение элемента массива по индексу;
- getLen(intarray arr) — возвращает длину массива;
- getArg(int index) — возвращает аргумент командной строки как массив кодов символов;
- readFile(intarray path) — читает файл по пути (массив символов), возвращает содержимое как массив кодов символов.

Данный этап является финальным в последовательности обработки исходного кода. Процесс интерпретации начинается с корневого узла дерева, во время которого рекурсивно выполняются все конструкции программы, изменяется состояние таблиц символов и в поток вывода выводится результат.

В ходе разработки интерпретатора были реализованы модульные тесты на языке Java, используя библиотеку JUnit5. Код тестов находятся в файле tests/main/InterpreterTests.java (Приложение Д). Тестируемые скрипты на языке aqua находятся в папке tests/files/.

Для избегания повторения кода в тестах реализованы две служебных функции. Функция test_success(String filename, Object var_value) загружает и запускает скрипт из файла, после интерпретации извлекает значение переменной res из глобальной области видимости и проверяет, что результат совпадает с ожидаемым. Функция test_fail(String filename, String error_msg) запускает скрипт и ожидает ошибку во время выполнения. При возникновении ошибки проверяет, что сообщение содержит ожидаемое сообщение.Тесты автоматически проверяют как успешные сценарии, так и обработку ошибок, ускоряя разработку программы.

2.5 Интеграция языка Aqua со средой разработки

Для повышения удобства разработки на языке Aqua была реализована интеграция со средой разработки Sublime Text [20]. Это позволило обеспечить базовую поддержку языка на уровне редактора и упростить процесс написания и запуска программ.

В рамках интеграции разработаны компоненты (Приложение Е):

- файл подсветки синтаксиса Aqua.sublime-syntax, реализующий разметку ключевых конструкций языка, включая ключевые слова, литералы, операторы и идентификаторы;
- файл настроек комментариев Aqua - Comments.tmPreferences, обеспечивающий корректную работу однострочных и многострочных комментариев;
- файл сборки Aqua.sublime-build, позволяющий запускать программы на языке Aqua непосредственно из среды разработки.

Файл Aqua.sublime-build интегрирует интерпретатор Aqua в стандартный механизм сборки Sublime Text, благодаря чему запуск программ осуществляется через встроенные средства среды без необходимости обращения к командной

строке. Поддерживается запуск для windows, linux и debug режим. Стоит отметить, что для корректной работы интерпретатор Aqua должен быть установлен в системе (Приложение Ж).

Реализованная интеграция обеспечивает базовую поддержку языка Aqua в редакторе: автоматическое распознавание “.aqua” файлов, подсветка синтаксиса, подсказки и система запуска программы. Это делает разрабатываемый язык более удобным и приближенным к использованию полноценных языков программирования.

Выводы по главе

В данной главе были обоснованы выборы инструментов и технологий для реализации интерпретатора языка Aqua, основными из которых стали Java, генераторы лексических анализаторов JFlex и синтаксических анализаторов BYACC/J. Разработана модульная архитектура интерпретатора, разделённая на фронтенд (анализ исходного кода) и рантайм (выполнение программы), что обеспечивает чёткое разделение ответственности между компонентами. Представлено описание языка Aqua, его синтаксиса, семантики, типов данных и управляющих конструкций, а также отмечены ключевые особенности. Реализованы лексический и синтаксический анализаторы, построено абстрактное синтаксическое дерево, на основе которого выполняются семантический анализ и интерпретация программы с использованием таблиц символов и обработчиков типов. Разработаны модульные тесты на базе JUnit5, автоматически проверяющие корректность выполнения успешных сценариев и обработку ошибок. Для повышения удобства разработки выполнена интеграция языка Aqua со средой Sublime Text, включающая подсветку синтаксиса, настройки комментариев и систему сборки. В совокупности реализованные компоненты представляют собой полноценный интерпретатор, пригодный для учебных и исследовательских целей, с поддержкой базовых средств разработки.

ЗАКЛЮЧЕНИЕ

В рамках данной работы была достигнута основная цель — анализ инструментов разработки языков программирования и создание интерпретатора учебного языка Aqua.

В теоретической части рассмотрены современные языки программирования, их особенности, а также принципы построения компиляторов и интерпретаторов. Отдельное внимание уделено архитектуре создания языков, включая этапы лексического, синтаксического и семантического анализа.

Обоснован выбор инструментов разработки. Использование Java в качестве основного языка реализации обеспечило переносимость, удобство разработки и наличие развитой экосистемы. Применение JFlex и BYACC/J позволило автоматизировать процессы генерации лексического и синтаксического анализаторов, что снизило вероятность ошибок и ускорило разработку. В практической части реализованы все основные компоненты интерпретатора:

- лексический анализатор, выполняющий разбиение входного потока на токены;
- синтаксический анализатор, строящий абстрактное синтаксическое дерево;
- семантический анализатор, проверяющий корректность программы;
- выполнение кода на основе абстрактного синтаксического дерева.

Разработан и формально описан язык программирования Aqua, определены его синтаксис, семантика, типовая система и управляющие конструкции. Реализована поддержка базовых типов данных, операторов управления потоком, функций, рекурсии и целочисленных массивов.

Проведено тестирование интерпретатора с использованием модульных тестов, что позволило подтвердить корректность работы как в штатных сценариях, так и при возникновении ошибок.

Реализована поддержка языка Aqua средой разработки Sublime Text. Разработаны файлы подсветки синтаксиса и сборки, обеспечивающие написание программ и их запуск непосредственно из редактора. Это повысило удобство разработки и продемонстрировало возможность интеграции языка с современным инструментом разработки, что приближает его к промышленным решениям.

Дополнительно стоит отметить, что в примерах программ на языке Aqua (Приложение И) продемонстрированы практические возможности языка: в файле examples/ege_task.aqua реализовано решение одной из задач ЕГЭ по информатике, а в файле examples/brain_fuck_interpreter.aqua реализован рабочий интерпретатор языка Brainfuck. Данные примеры подтверждают достаточный уровень абстракции и полноту реализованных возможностей языка.

Оценка производительности языка

Для оценки производительности интерпретатора языка Aqua было проведено экспериментальное сравнение времени выполнения программы examples/ege_task.aqua с аналогичной реализацией на языке Python. Измерения показали, что среднее время выполнения составило: 164 мс для языка Aqua и 38 мс для Python. Таким образом, интерпретатор Aqua уступает по скорости выполнения примерно в 4 раза.

Полученный результат является ожидаемым и объясняется рядом факторов: отсутствием оптимизаций, характерных для зрелых интерпретаторов, а также учебной направленностью реализации. При этом достигнутый уровень производительности можно считать удовлетворительным, поскольку он подтверждает корректность архитектурных решений и работоспособность интерпретатора.

Перспективными направлениями для дальнейшего развития являются: реализация более глубокого анализа корректности программ и поиска потенциальных ошибок; расширение типовой системы; улучшение механизмов обработки ошибок; добавление новых структур данных и языковых конструкций; внедрение в инфраструктуру современных сред разработки.

Отдельный интерес представляет переход от интерпретации к компиляции с использованием таких инструментов, как LLVM [21]. Это позволит повысить производительность и приблизить язык к промышленным решениям.

В ходе работы был выполнен комплексный анализ подходов к разработке языков программирования и реализован полноценный интерпретатор учебного языка. Практическая реализация показала, что современные инструменты позволяют эффективно создавать языки программирования без необходимости ручной реализации всех компонентов.

Разработанный язык Aqua демонстрирует баланс между простотой и выразительностью и может использоваться в образовательных целях для изучения принципов построения языков.

Полученные результаты подтверждают применимость выбранных подходов и инструментов, а также могут служить основой для дальнейших исследований.

Разработка интерпретатора продемонстрировала навыки работы с формальными языками, алгоритмами, архитектурой программных систем, а также способность реализовывать сложные программные решения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Memory management library [Электронный ресурс] // cppreference. – URL: https://en.cppreference.com/w/cpp/memory.html (дата обращения: 07.04.2026).

2. Управление памятью: Взгляд изнутри [Электронный ресурс] // Хабр. – URL: https://habr.com/ru/articles/270009/ (дата обращения: 15.04.2026).

3. Плюсы и минусы Java [Электронный ресурс] // Си Ай Скул. – URL: https://www.cischool.ru/plyusy-i-minusy-java/ (дата обращения: 03.04.2026).

4. Плюсы и минусы языка программирования Java [Электронный ресурс] // zdrons. – URL: https://zdrons.ru/veb-programmirovanie/pljusy-i-minusy-yazyka-programmirovaniya-java/ (дата обращения: 12.04.2026).

5. Execution model [Электронный ресурс] // Python 3 documentation. – URL: https://docs.python.org/3/reference/executionmodel.html (дата обращения: 19.04.2026).

6. Python: компилируемый или интерпретируемый язык, правда скрыта [Электронный ресурс] // Sky.pro. – URL: https://sky.pro/wiki/python/python-kompiliruemyj-ili-interpretiruemyj-yazyk/ (дата обращения: 09.04.2026).

7. mypy – Optional Static Typing for Python [Электронный ресурс] // mypy-lang. – URL: https://mypy-lang.org/ (дата обращения: 14.04.2026).

8. Deven, J. Goroutines in Go: A Practical Guide to Concurrency [Электронный ресурс] / J. Deven // getstream. – 2025. – URL: https://getstream.io/blog/goroutines-go-concurrency-guide/ (дата обращения: 05.04.2026).

9. Rust Memory Safety 2026: 82% Better Performance Proven [Электронный ресурс] // byteiota. – URL: https://byteiota.com/rust-memory-safety-2026-82-better-performance-proven/ (дата обращения: 18.04.2026).

10. Memory Safety in Rust [Электронный ресурс] // AdaCore Blog. – 2024. – URL: https://www.adacore.com/blog/memory-safety-in-rust (дата обращения: 02.04.2026).

11. Ахо, А. В. Компиляторы: принципы, технологии и инструментарий / А. В. Ахо, М. с. Лам, Р. Сети, Д. Д. Ульман ; пер. с англ. – 2-е изд. – М. : ООО «И.Д. Вильямс», 2018. – 1184 с. – ISBN 978-5-8459-1932-8.

12. Джеффери, К. Л. Создай свой собственный язык программирования / К. Л. Джеффери ; пер. с англ. с. В. Минца. – М. : ДМК Пресс, 2022. – 408 с.

13. Java Documentation [Электронный ресурс] // Oracle. – URL: https://docs.oracle.com/en/java/ (дата обращения: 11.04.2026).

14. IntelliJ IDEA: Ведущая IDE для профессиональной разработки на Java и Kotlin [Электронный ресурс] // JetBrains. – URL: https://www.jetbrains.com/ru-ru/idea/ (дата обращения: 16.04.2026).

15. JFlex [Электронный ресурс] // jflex. – URL: https://jflex.de/ (дата обращения: 08.04.2026).

16. Lex (software) [Электронный ресурс] // Wikipedia. – URL: https://en.wikipedia.org/wiki/Lex_(software) (дата обращения: 20.04.2026).

17. What is ANTLR? [Электронный ресурс] // ANTLR – URL: https://www.antlr.org/ (дата обращения: 04.04.2026).

18. BYacc/J [Электронный ресурс] // SourceForge. – URL: https://byaccj.sourceforge.net/ (дата обращения: 13.04.2026).

19. Yacc [Электронный ресурс] // Википедия. – URL: https://ru.wikipedia.org/wiki/Yacc (дата обращения: 06.04.2026).

20. Sublime Text Documentation [Электронный ресурс] // sublimetext – URL: https://www.sublimetext.com/docs/ (дата обращения: 17.04.2026).

21. The LLVM Compiler Infrastructure [Электронный ресурс] // llvm – URL: https://llvm.org/ (дата обращения: 01.04.2026).

ПРИЛОЖЕНИЯ

Приложение А

Фрагменты исходного кода модуля frontend

Исходный код классов модуля frontend доступен по ссылке: https://github.com/vankad24/aqua_lang_interpreter/tree/master/src/frontend

Приложение Б

Фрагменты исходного кода модуля runtime

Исходный код классов модуля runtime доступен по ссылке: https://github.com/vankad24/aqua_lang_interpreter/tree/master/src/runtime

Приложение В

Примеры программ на языке Aqua

Простейшая программа:

```
println("Hello, World!")
```

Условный оператор:

```
int temperature = 30
if temperature >= 0 {
   println("Тепло")
} else {
   println("Холодно")
}
```

Цикл for:

```
for i : 5 {
   println(i)
}
```

Функция (рекурсия):

```
fn factorial(int n) {
   if n <= 1 {
      return 1
   }
   return n * factorial(n - 1)
}
println(factorial(5))
```

Работа с массивами:

```
intarray arr = newIntArray(5)
for i : getLen(arr) {
   setElement(arr, i, i * 2)
}
println(arr)
```

Приложение Г

Консольные команды для запуска утилит

Консольные команды для генерации кода с помощью утилит JFlex и BYACC/J. Примечание: данные команды выполняются на операционной системе windows из корневой папки проекта.

Генерация лексера:

```
cd .\src\frontend\
..\..\jflex-1.9.1\bin\jflex .\javalex.l
```

Генерация парсера:

```
cd .\src\frontend\
..\..\byaccj1.15_win32\yacc.exe -Jpackage=frontend -Jclass=Parser j0gram.y
```

Приложение Д

Реализация тестирования

Исходный код файлов классов модуля tests доступен по ссылке: https://github.com/vankad24/aqua_lang_interpreter/tree/master/tests

Приложение Е

Интеграция со средой разработки

Исходный код файлов директории SublimePackage доступен по ссылке: https://github.com/vankad24/aqua_lang_interpreter/tree/master/SublimePackage

Приложение Ж

Установка языка Aqua

Для запуска интерпретатора необходимо выполнить сборку проекта или скачать.jar файл по ссылке (https://github.com/vankad24/aqua_lang_interpreter/releases), а затем поместить вместе с скриптом запуска. Для windows это скрипт “aqua.bat”, для linux – “aqua”. Далее необходимо добавить эту папку в переменные среды *Path* (уточните способ для вашей системы). Теперь команда aqua должна быть доступна из командной строки.

Приложение И

Практические примеры использования

Исходный код файлов директории examples доступен по ссылке: https://github.com/vankad24/aqua_lang_interpreter/tree/master/examples